ARE SELF-EVOLVING AGENT SKILLS LEARNING SKILLS?

Anonymous Submission

ABSTRACT

Self-evolving agents store experience in persistent skills, but a higher task score leaves open what the skill has learned. We study whether the resulting benefit persists beyond the source task and which decisions in the skill explain its success or failure. We diagnose completed SkillEvolver, SPARK, and CoEvoSkills artifacts through repeated paired evaluations, controlled task variations, and inspection of skill contents and execution traces, using their construction procedures as context. Source-task gains recur across all three methods, while changed tasks expose different scopes of useful reuse. Contrasting cases relate reuse outcomes to how skills distinguish valid answers from plausible candidates and preserve the constraints that define a correct solution. The findings connect benchmark gains to persistent decision guidance and distinguish producing a useful skill, applying its guidance, and sustaining its benefit under task change.

***Index Terms*—** Agent skills, self-evolution, diagnostic evaluation, skill reuse, generalization

1. INTRODUCTION

Persistent agent skills offer a route to improvement without updating model parameters. Instructions, scripts, and supporting references make execution experience available to later runs. SkillEvolver, SPARK, and CoEvoSkills construct these artifacts through deployment-grounded revision, verified-trajectory distillation, and co-evolutionary verification [1–3]. Their outputs motivate the question in our title: when a system produces a higher-scoring skill, what useful behavior has become persistent?

The location of that improvement matters. A skill can preserve a decision rule that remains useful on new inputs, a choice tailored to the original instance, or a check that a later executor fails to apply. These artifacts may all help on the source task, yet lead to different outcomes when the task changes. The final score also combines artifact construction, execution, and verification. Understanding skill learning therefore requires following the stored guidance into the behavior it supports.

We ask two connected questions. **Does the benefit persist, and under which task changes?** Repeated source-task evaluations test whether paired utility recurs, while frozen-skill variants expose the demands that the same guidance can accommodate. **What in the skill and its use explains the observed outcomes?** Contrasting executions reveal whether decisive correctness criteria are available in the skill and applied to the new task. These questions connect the existence and scope of useful behavior with an account of how it is produced.

We examine three construction mechanisms through two standard-task runs per method and controlled changes to input representation, task instances, and coupled requirements. Each skill condition is paired with a no-skill execution of the same task. Signal-detection and scheduling cases then connect the aggregate patterns to decisions visible in the skills and execution traces. The completed artifact is the common object of diagnosis: its construction provides context, and its later use supplies the behavioral evidence.

All three methods yield positive mean source-task gains in both runs, but this agreement does not persist under task variation. SkillEvolver remains positive across all three change types. SPARK's benefit depends on the change type, and CoEvoSkills has negative mean gains in all three. The clearest separation occurs when the instance is replaced. The cases explain why executing a procedure is only part of the problem: the executor must also distinguish the required signal from misleading candidates and a valid schedule repair from an otherwise feasible schedule.

Our contribution is a diagnosis of what observed skill gains represent. Repeated paired evaluations establish recurring utility, controlled variants locate its scope, and artifact-level cases identify decisions associated with successful and failed reuse. Together, these results show how a score advantage can be interpreted through the guidance that survives construction and is actually used during execution.

2. PROBLEM FORMULATION

2.1. Skill Revision and Evaluation

At revision round r, an optimizer updates the current persistent skill artifact:

$$s_{r+1} = O(s_r, \tau_r, f_r), \tag{1}$$

where s_r is the current skill, τ_r is an execution trace, and f_r is feedback from execution and evaluation. The operator O denotes the method's complete update pathway, including its native merge, validation, selection, or adoption mechanism. A generated candidate that is rejected or never adopted is not s_{r+1}.

The agent then executes task x with the frozen skill:

$$o_r = A(x, s_r, z_r), \tag{2}$$

where z_r represents model sampling, tool use, and environment state, and o_r is the resulting output artifact or state change. A verifier maps the output to a score:

$$y_r = V(x, o_r). \tag{3}$$

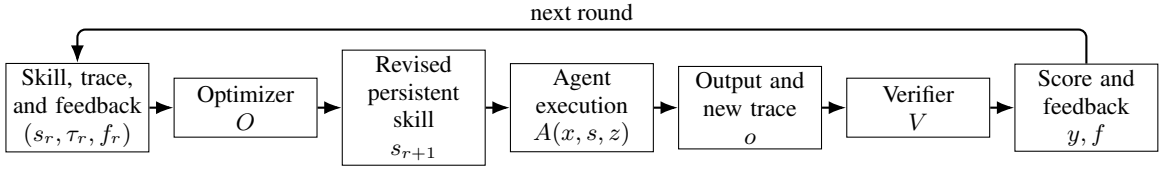


Fig. 1. The skill-evolution lifecycle. Construction produces a persistent artifact; deployment turns its guidance into behavior; verification assigns the observed score. We connect these stages through paired utility, frozen-skill task variation, and inspection of skill use.

2.2. From Score Gains to Skill Behavior

The score $V(x, A(x, s, z))$ records the outcome of a skill used by an executor and assessed by a verifier. Comparing a completed skill with no skill measures its utility at deployment. Comparing the adopted final skill with its actual parent addresses the improvement associated with revision. The experiments here follow the first contrast across repeated runs and changed tasks.

For each source task x, we freeze its completed skill s_x and evaluate it on a variant $T_k(x)$ alongside a no-skill execution. The skill remains unchanged, while the executor can reinterpret the input, recompute a solution, and adapt its actions. A positive paired difference on the variant indicates that the stored guidance remains useful under that change. The variation types distinguish the decisions that can be reused from those that the executor must reconsider.

The attribution question follows the same artifact into execution. We examine which candidates its rules distinguish, which constraints its checks preserve, and where the observed behavior departs from that guidance. The construction procedures describe the feedback from which the guidance is produced. Reading these stages together connects a score change to a specific decision, rather than treating the presence of a skill file as evidence that its instructions were used effectively.

3. DOES SKILL UTILITY PERSIST BEYOND THE SOURCE TASK?

We first test whether completed skills provide a recurring benefit on their source tasks. We then change the task while keeping its skill frozen to determine which demands that benefit survives. The paired no-skill execution provides the reference for both comparisons.

3.1. Experimental Setup

We evaluate SkillEvolver [1], SPARK [2], and CoEvoSkills [3] on SkillsBench [4], with two runs per method. Each run pairs the method's skill condition with a no-skill execution of the same task. Both runs of each method use the same task set: 81 tasks for SkillEvolver, 84 for SPARK, and 85 for CoEvoSkills.

We report mean verifier reward on a 0–100 scale and the mean within-task difference between the skill and no-skill conditions. For a run with N tasks, the paired gain is

$$\Delta = \frac{100}{N} \sum_{i=1}^{N} \left(r_i^{\text{skill}} - r_i^{\text{no-skill}} \right). \tag{4}$$

We also count tasks with positive, negative, and zero differences.

3.2. Positive Source-Task Utility in Both Runs

All three methods have positive mean paired gains in both runs (Table 1). SkillEvolver gains 25.61 and 25.02 points, CoEvoSkills gains 10.24 and 9.76, and SPARK gains 1.84 and 6.45. SkillEvolver and CoEvoSkills retain similar gains despite substantial changes in their no-skill scores. The paired comparison reveals recurring utility that absolute skill-condition scores alone would obscure.

The task-level results also differ. SkillEvolver has at least four times as many improvements as regressions in each run; CoEvoSkills improves 17 tasks in each run, with nine and ten regressions. SPARK's first run is nearly balanced, while its second has 13 improvements and five regressions. The gain therefore reflects both the size and frequency of task-level changes. The frozen-skill variants test whether the useful guidance behind these source-task outcomes survives a different task demand.

Method	Run	N	No skill	Skill	Δ	I/R/T
SkillEvolver	1	81	30.20	55.81	$+25.61$	27/6/48
	2	81	37.32	62.34	$+25.02$	26/3/52
SPARK	1	84	17.82	19.66	$+1.84$	9/8/67
	2	84	14.37	20.81	$+6.45$	13/5/66
CoEvoSkills	1	85	42.21	52.44	$+10.24$	17/9/59
	2	85	23.54	33.31	$+9.76$	17/10/58

Table 1. Two runs per method on SkillsBench. Mean verifier rewards and paired gain Δ use a 0–100 scale. I/R/T counts improved, regressed, and tied tasks.

3.3. Task Variation Tests What Remains Useful

We apply each source-task skill to controlled variants without further evolution. Input-representation changes (P1) alter names, column arrangements, or schemas; task-instance changes (P2) replace the data or instance structure; coupled-requirement changes (P3) alter or add requirements that must be satisfied together. The corresponding decision demands are to remap the input, recompute a solution for a replacement instance, and accommodate changed correctness requirements. A frozen skill must support these adaptations through guidance already present in the artifact.

The complete record contains 222 variants and 886 condition-level results across no skill and the three methods. Table 2 summarizes historical and recovery evaluations for 222 SkillEvolver variants and 221 each for SPARK and CoEvoSkills, pairing each skill condition with the no-skill result on the same variant.

Source-task improvement does not imply useful generalization. All three methods improve mean reward on the source tasks in both runs, but their variant results diverge. SkillEvolver retains a positive mean gain of 11.94 points, SPARK is close to neutral at $+0.65$, and CoEvoSkills falls below the no-skill baseline at -5.01. SkillEvolver also improves more variants than it regresses (47 versus 20), whereas SPARK and CoEvoSkills show the reverse. The improvement and regression counts clarify the near-zero SPARK mean: it combines changes in opposite directions, rather than uniformly neg-

ligible effects. SkillEvolver shows both a larger positive mean and more frequent improvement than regression.

Replacing the instance produces the largest divergence. SkillEvolver remains positive under all three changes, with its largest gain on P2 ($+14.56$). P2 is also where SPARK and CoEvoSkills decline most (-5.23 and -9.51). SPARK retains positive averages under representation and requirement changes, but loses its advantage when the underlying instance changes. P2 requires choices to be recomputed from new data or a new instance structure. A procedure that remains useful after renaming inputs can therefore lose its benefit when the underlying decision changes. SkillEvolver's positive gains across P1–P3 show the broadest observed persistence of utility, while CoEvoSkills' negative means show that source-task guidance can become counterproductive after the task changes.

A. Overall utility on task variants					
Method	N	**No skill**	**Skill**	Δ	**I/R/T**
SkillEvolver	222	41.87	53.81	$+11.94$	47/20/155
SPARK	221	41.61	42.25	$+0.65$	32/37/152
CoEvoSkills	221	41.61	36.60	-5.01	26/41/154

B. Utility by task change: $(N)\ \Delta$			
Method	**P1**	**P2**	**P3**
SkillEvolver	$(77)\ +11.85$	$(68)\ +14.56$	$(77)\ +9.72$
SPARK	$(77)\ +2.00$	$(68)\ -5.23$	$(76)\ +4.53$
CoEvoSkills	$(77)\ -3.59$	$(68)\ -9.51$	$(76)\ -2.42$

Table 2. Skill utility under task variation: descriptive summary of historical and recovery evaluations. Each method is paired with its same-variant no-skill baseline. P1/P2/P3 denote representation, instance, and requirement changes. Panel B gives variant count N and paired gain Δ; scores and I/R/T follow Table 1.

4. LOCATING SUCCESSFUL AND FAILED REUSE

The aggregate results locate differences in scope; the cases locate the decisions behind them. Signal detection tests how a skill distinguishes the required signal from plausible candidates. Scheduling tests whether a skill preserves the conditions that define an acceptable repair. In each case, we relate the rules in the artifact to their observed use.

4.1. Distinguishing a Candidate from a Valid Answer

In exoplanet detection, the SkillEvolver skill supports successful detection after input columns are rearranged (P1) and after the task requires two signals (P3). The CoEvoSkills skill fails both variants: its use leads to stellar modulation being reported as the target signal, and a signal and its harmonic being treated as two separate signals. Both skills fail the replacement-instance P2 variant.

The difference is visible in what the skills ask the executor to check. SkillEvolver separates candidate discovery, signal diagnosis, and refinement. Its guidance addresses misleading peaks, transit-like durations, and harmonic relationships. CoEvoSkills also includes validation, including phase-fold, odd/even, and residual checks, but its use in these variants does not resolve the misleading candidates. The presence of a checking routine is therefore less informative than whether its criteria distinguish the alternatives that matter on the changed task. SkillEvolver's P1 and P3 successes show this guidance being useful under changed inputs and requirements. Its P2 failure locates a different boundary within the same task family: the retained criteria do not resolve that replacement instance, despite a positive P2 average across the broader evaluation.

4.2. Preserving the Rules That Define a Correct Repair

In scheduling, the SkillEvolver and SPARK skills succeed on all three variants; the CoEvoSkills skill succeeds only on P3. In P1, using CoEvoSkills yields an otherwise feasible schedule that fails a required local-minimality check on the baseline-relative repair. Feasibility checks alone do not capture this local-minimality requirement.

The SkillEvolver skill explicitly retains local minimality and baseline-relative repair rules. These rules specify how an acceptable repair differs from an arbitrary feasible schedule, and remain relevant after the input names or scheduling instance change. The CoEvoSkills skill covers several structural and feasibility checks, yet the failed deployment misses the decisive repair condition. The contrast concerns whether the decisive repair condition is preserved in guidance and enforced during its use. SPARK's success across the same three variants also shows that this form of successful reuse is not exclusive to one method.

4.3. Construction Feedback and Skill Content

The construction procedures help interpret how these decision criteria enter a skill. SkillEvolver compares successful and failed deployments of a candidate skill and patches missing or misleading guidance. Its independent audit also checks reliance on training-instance constants [1]. This exposes failures in using the skill and encourages rules that a later executor can apply to a different input. The scheduling and signal-detection skills reflect that emphasis by retaining criteria for checking a repair or discriminating between candidate signals.

SPARK grounds skill distillation in environment-verified trajectories, preserving execution evidence, checks, and lessons [2]. Its scheduling result shows that such evidence can support successful reuse. Its mixed aggregate profile raises a separate question about abstraction: a verified execution supplies evidence for what worked on the source task, while changed instances require determining which choices should be retained and which should be recomputed. The observed decline on P2 makes that distinction especially relevant to interpreting its results.

CoEvoSkills obtains detailed revision feedback from a co-evolving surrogate verifier, while the ground-truth oracle returns an opaque pass/fail signal [3]. This makes the surrogate's coverage of correctness conditions relevant to the guidance retained in the skill. This provides a plausible account of the scheduling failure: the skill preserves many checks, but successful reuse still depends on a condition those checks did not enforce in that deployment. Across the cases, feedback coverage and abstraction from the source instance are candidate explanations for the different scopes of utility.

5. INTERPRETING GAINS AS EVIDENCE OF SKILL LEARNING

5.1. A Useful Source Skill Can Have a Narrow Behavioral Scope

The standard-task and variant results identify two properties of the completed artifacts. Source-task utility recurs for all three methods, but its scope differs under controlled changes. SkillEvolver's positive gains across P1–P3 support useful reuse under each tested

change type. SPARK's positive P1 and P3 means alongside a negative P2 mean locate a more specific difficulty: adapting the representation or requirements can preserve its benefit, while replacing the underlying instance can remove it. CoEvoSkills' negative means show that guidance useful on the source task can interfere with a changed task.

Aggregate scope also contains local boundaries. SkillEvolver has the strongest P2 mean yet fails the replacement-instance signal-detection case. SPARK is nearly neutral across variants but succeeds on all three scheduling changes. These observations locate success and failure at the interaction between a skill and a task demand. The aggregate comparison reveals where to investigate; the case explains which decision remains useful or breaks.

5.2. Correctness Criteria Must Survive Construction and Use

The two cases expose distinct points where reuse can break. In signal detection, a procedure produces plausible candidates, but success depends on identifying which candidate satisfies the task. In scheduling, an output can satisfy structural feasibility while violating the rule that defines an acceptable repair. Reusable guidance must preserve this relationship between the procedure and its correctness criteria. The procedure's availability alone does not ensure that a later executor makes the required distinction.

This explains why inspecting the artifact and its execution together matters. CoEvoSkills contains validation checks, yet the observed executions miss decisive conditions. SkillEvolver's successful cases combine explicit decision criteria with their application to changed inputs. The contrast is between guidance as written and guidance as used: retained checks contribute only when they cover the relevant condition and are enforced during execution.

The lifecycle perspective also makes feedback informative. Deployment failures can reveal where instructions are ambiguous, verified trajectories can supply evidence for a successful procedure, and surrogate feedback can identify conditions to check. Each construction mechanism must convert that evidence into guidance that a later executor can apply. Our results locate the empirical question at this conversion and subsequent use: which decisions are preserved, when do they remain appropriate, and what happens when the executor follows them? This links the reported score back to the behavior that a persistent skill is intended to improve.

6. RELATED WORK

Learning persistent skills from experience. SkillEvolver, SPARK, and CoEvoSkills make experience persistent through different construction loops: deployment-grounded revision, verified-trajectory distillation, and co-evolutionary verification, respectively [1–3]. These methods motivate our focus on the artifact delivered to a later executor. We study how that artifact's usefulness changes after its source task is varied, and interpret the resulting behavior through its retained guidance.

Evaluating skill utility and use. SkillsBench evaluates skill utility across diverse tasks, while SWE-Skills-Bench studies skill use in software engineering [4, 5]. SkillLearnBench evaluates continual skill generation on real-world tasks [6]. From Raw Experience to Skill Consumption separates experience generation, skill extraction, and downstream consumption [7]. Our study connects these evaluation levels to the identification of skill learning: repeated outcomes characterize completed-skill utility, task changes test its persistence, and skill-level examples explain the decisions behind contrasting outcomes. The unit of interpretation is the persistent artifact in use.

7. CONCLUSION

Our diagnosis connects score gains from self-evolving skills to the behavior carried by their persistent artifacts. All three methods provide recurring source-task utility, while controlled task changes reveal sharply different scopes of reuse. SkillEvolver sustains positive average gains across representation, instance, and requirement changes; the other methods show more selective or adverse reuse. Contrasting cases locate the difference in decisions that distinguish valid answers and preserve repair constraints, including whether the executor applies the checks present in a skill. These results make the learning question concrete: experience must become useful guidance, and that guidance must continue to support correct decisions when the task changes. Evaluating skill learning therefore requires connecting construction, stored content, and observed use, with the benchmark score recording the outcome of that process.

8. REFERENCES

[1] Genrui Zhang, Erle Zhu, Jinfeng Zhou, Caiyan Jia, and Hongning Wang, "SkillEvolver: Skill Learning as a Meta-Skill," 2026.

[2] Yang Zhou, Zihan Dong, Zhenting Wang, Can Jin, Shiyu Zhao, Bangwei Guo, Difei Gu, Linjun Zhang, Mu Zhou, and Dimitris N. Metaxas, "Evidence Over Plans: Online Trajectory Verification for Skill Distillation," 2026.

[3] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, Xue Liu, Xiaoxiao Li, and Philip S. Yu, "CoEvoSkills: Self-Evolving Agent Skills via Co-Evolutionary Verification," 2026.

[4] Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, Shenghan Zheng, Kyoung Whan Choe, Jiankai Sun, Shuyi Wang, Chujun Tao, Binxu Li, Xuandong Zhao, Hejia Geng, Xiaojun Wu, Junwei Zhou, Xiaokun Chen, Hanwen Xing, Yubo Li, Qunhong Zeng, Di Wang, Yuanli Wang, Roey Ben Chaim, Penghao Jiang, Haotian Shen, Luyang Kong, Xinyi Liu, Runhui Wang, Xuanqing Liu, Jiachen Li, Xin Lan, Yueqian Lin, Wengao Ye, Junwei He, Songlin Li, Yue Zhang, Yipeng Gao, Yijiang Li, Ze Ma, Liqiang Jing, Tianyu Wang, Kaixin Li, Yiqi Xue, Haoran Lyu, Yizhuo He, Yuchen Tian, Shutong Wu, Bowei Wang, Yixuan Gao, Bo Chen, Litong Liu, Sikai Cheng, Jiajun Bao, Shuaicheng Tong, Shuwen Xu, Terry Yue Zhuo, Tinghan Ye, Qi Qi, Miao Li, Longtai Liao, Zelin Tan, Chang Shi, Xilin Tang, Srinath Tankasala, Boqin Yuan, Yaoyao Qian, Jianhong Tu, Chenguang Wang, Yizhou Sun, Wei Wang, Aaron Taylor, Ziyue Yang, Changkun Guan, Zhikang Dong, Xinyu Zhang, Steven Dillmann, Han chung Lee, and Dawn Song, "SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks," 2026.

[5] Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu, "SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering?," 2026.

[6] Shanshan Zhong, Yi Lu, Jingjie Ning, Yibing Wan, Lihan Feng, Yuyi Ao, Leonardo F. R. Ribeiro, Markus Dreyer, Sean Ammirati, and Chenyan Xiong, "SkillLearnBench: Benchmarking Continual Learning Methods for Agent Skill Generation on Real-World Tasks," 2026.

[7] Zisu Huang, Jingwen Xu, Yifan Yang, Ziyang Gong, Qihao Yang, Muzhao Tian, Xiaohua Wang, Changze Lv, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Xue Yang, Dongdong Chen, Xiaoqing Zheng, and Chong Luo, "From Raw Experience to Skill Consumption: A Systematic Study of Model-Generated Agent Skills," 2026.